

CSE110A: Compilers

Topics:

- *LVN Memory*
- *LVN Functions*

Local Value Numbering: Memory

Local value numbering: Memory

- Consider a 3 address code that allows memory accesses

```
a[i] = x[j] + y[k];  
b[i] = x[j] + y[k];
```

is this transformation allowed?

```
a[i] = x[j] + y[k];  
b[i] = a[i];
```

Local value numbering: Memory

- Consider a 3 address code that allows memory accesses

```
a[i] = x[j] + y[k];  
b[i] = x[j] + y[k];
```

is this transformation allowed?
No!

```
a[i] = x[j] + y[k];  
b[i] = a[i];
```

only if the compiler can prove that *a* does not alias *x* and *y*

In the worst case, every time a memory location is updated, the compiler must update the value for all pointers.

Local value numbering: Memory

- Consider a 3 address code that allows memory accesses

```
a[i] = x[j] + y[k];  
b[i] = x[j] + y[k];
```



```
a[i] = x[j] + y[k];  
b[i] = a[i];
```

Example, initially:

```
i = j  
a = x  
y[k] = 1  
x[j] = 1
```

What does b[i] equal at the end of each computation?

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair

```
a[i] = x[j] + y[k];  
b = x[j] + y[k];
```

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b, 6) = (x[j], ?) + (y[k], ?);
```

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b, 6) = (x[j], 4) + (y[k], 5);
```

Does this help at all?

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b, 6) = (x[j], 4) + (y[k], 5);  
(c, 7) = (x[j], 4) + (y[k], 5);
```

Does this help at all?

If there is no memory writes between an assignment to a variable then we can do a replacement

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b, 6) = (x[j], 4) + (y[k], 5);  
(c, 7) = (b, 6);
```

Does this help at all?

If there is no memory writes between an assignment to a variable then we can do a replacement

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b[i], 6) = (x[j], 4) + (y[k], 5);
```

A compiler analysis might try to determine that addresses can't alias

can we trace a, x, y to

```
a = malloc(...);  
x = malloc(...);  
y = malloc(...);
```

// a, x, y are never overwritten

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b[i], 6) = (x[j], 1) + (y[k], 2);
```

in this case we do not have to update the number

A compiler analysis might try to determine that addresses can't alias

can we trace a, x, y to

```
a = malloc(...);  
x = malloc(...);  
y = malloc(...);
```

// a, x, y are never overwritten

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b[i], 6) = (x[j], 4) + (y[k], 5);
```

programmer annotations can also tell the compiler that no other pointer can access the memory pointed to by a

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b[i], 6) = (x[j], 4) + (y[k], 5);
```

in this case we do not have to update the number

`restrict a`

programmer annotations can also tell the compiler that no other pointer can access the memory pointed to by a

Warning: the compiler does not enforce this!

Local value numbering: Memory

- How to number:
 - Number each pointer/index pair
 - Any pointer/index pair that might alias must be incremented at each instruction

```
(a[i], 3) = (x[j], 1) + (y[k], 2);  
(b[i], 6) = (a[i], 3);
```

Local Value Numbering: Functions

Local value numbering: functions

How to number?

```
a = foo(x) ;  
x = b ;  
c = foo(x) ;
```

Local value numbering: functions

How to number?

```
a = foo(x) ;  
x = b ;  
c = foo(x) ;
```

the same way

```
a1 = foo(x0) ;  
x3 = b2 ;  
c4 = foo(x3) ;
```

What if you had
first class functions?

Local value numbering: functions

How to number?

```
a = foo(x) ;  
x = b ;  
c = foo(x) ;
```

the same way

```
a1 = foo(x0) ;  
x3 = b2 ;  
c4 = foo(x3) ;
```

Can we replace?

Local value numbering: functions

How to number?

```
a = foo(x);  
c = foo(x);
```

the same way

```
a1 = foo(x0);  
c2 = foo(x0);
```

How about now?

Local value numbering: functions

How to number?

```
a = foo(x);  
c = foo(x);
```

the same way

```
a1 = foo(x0);  
c2 = foo(x0);
```

How about now?

```
int count = 0;  
int foo(int x) {  
    count += 1;  
    return 0;  
};
```

What if foo had
this implementation?

Local value numbering: functions

How to number?

```
a = foo(x);  
c = foo(x);
```

the same way

```
a1 = foo(x0);  
c2 = foo(x0);
```

How about now?

side effects!

```
int count = 0;  
int foo(int x) {  
    count += 1;  
    return 0;  
};
```

What if foo had
this implementation?

Local value numbering: functions

```
a = foo(x);  
c = foo(x);  
print(count);
```

```
a1 = foo(x0);  
c2 = a1;  
print(count);
```

are these two programs the same?

```
int count = 0;  
int foo(int x) {  
    count += 1;  
    return 0;  
};
```

Local value numbering: functions

- In C/++, functions are assumed to have side effects
- A function that does not have side effects is called “pure”
 - You can annotate a function as pure
 - `__attribute__((pure))`
 - **warning**: compiler does not check this and you can introduce subtle bugs
- Functional languages tend to have a pure-by-default design. Allows more compiler optimizations, but less control to the programmer.